



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ
НАУКА У НОВОМ САДУ



Марина Ивановић

**Имплементација
дистрибуираног система за
колаборацију у реалном
времену заснованог на CRDT
структурама**

ЗАВРШНИ РАД

Основне академске студије

Нови Сад, 2026

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Број:
	ЗАДАТАК ЗА ЗАВРШНИ РАД	Датум:

(Податке уноси предметни наставник - ментор)

Студијски програм:	Софтверско инжењерство и информационе технологије		
Студент:	Марина Ивановић	Број индекса:	SV6/2022
Степен и врста студија:	Основне академске студије		
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	Игор Дејановић		
НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА: - проблем – тема рада; - начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;			

НАСЛОВ ЗАВРШНОГ РАДА:

Имплементација дистрибуираног система за колаборацију у реалном времену заснованог на CRDT структурама
--

ТЕКСТ ЗАДАТКА:

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim aenean sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim aenean sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
--

Руководилац студијског програма:	Ментор рада:

Примерак за: ☐ - Студента; ☐ - Ментора
--

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6
	КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР:	
Идентификациони број, ИБР:	
Тип документације, ТД:	Монографска документација
Тип записа, ТЗ:	Текстуални штампани материјал
Врста рада, ВР:	Дипломски - бечелор рад
Аутор, АУ:	Марина Ивановић
Ментор, МН:	Др Игор Дејановић, редовни професор
Наслов рада, НР:	Имплементација дистрибуираног система за колаборацију у реалном времену заснованог на CRDT структурама
Језик публикације, ЈП:	српски/хирилица
Језик извода, ЈИ:	српски/енглески
Земља публиковања, ЗП:	Република Србија
Уже географско подручје, УГП:	Војводина
Година, ГО:	2026
Издавач, ИЗ:	Ауторски репринт
Место и адреса, МА:	Нови Сад, трг Доситеја Обрадовића 6
Физички опис рада, ФО: (поглавља/страница/ цртања/табела/слика/графика/прилога)	7/48/16/8/14/0/2
Научна област, НО:	Електротехничко и рачунарско инжењерство
Научна дисциплина, НД:	Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО:	Дистрибуирани системи, колаборативно уређивање, CRDT, RGA, реално време, микросервиси, WebSocket
УДК	
Чува се, ЧУ:	У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, ВН:	
Извод, ИЗ:	У раду је приказано пројектовање и имплементација дистрибуираног система CodeStream за колаборативно уређивање програмског кода у реалном времену. Конфликти услед истовремених измена решавају се CRDT структуром података заснованом на RGA алгоритму. Систем чини скуп независних Rust микросервиса који комуницирају синхронно (HTTP) и асинхронно (RabbitMQ), уз JWT аутентификацију, синхронизацију стања документа преко Redis-а и изоловано извршавање кода унутар Docker контејнера.
Датум прихватања теме, ДП:	
Датум одбране, ДО:	01.01.2025
Чланови комисије, КО:	Председник: Др Петар Петровић, ванредни професор
	Члан: Др Марко Марковић, доцент
	Члан:
	Члан, ментор: Др Игор Дејановић, редовни професор
	Потпис ментора

	UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES 21000 NOVI SAD, Trg Dositeja Obradovića 6	
	KEY WORDS DOCUMENTATION	
Accession number, ANO :		
Identification number, INO :		
Document type, DT :	Monographic publication	
Type of record, TR :	Textual printed material	
Contents code, CC :		
Author, AU :	Marina Ivanović	
Mentor, MN :	Igor Dejanović, Phd., full professor	
Title, TI :	Implementation of a Distributed Real-Time Collaboration System Based on CRDT Structures	
Language of text, LT :	Serbian	
Language of abstract, LA :	Serbian/English	
Country of publication, CP :	Republic of Serbia	
Locality of publication, LP :	Vojvodina	
Publication year, PY :	2026	
Publisher, PB :	Author's reprint	
Publication place, PP :	Novi Sad, Dositeja Obradovica sq. 6	
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	7/48/16/8/14/0/2	
Scientific field, SF :	Electrical and Computer Engineering	
Scientific discipline, SD :	Applied computer science and informatics	
Subject/Key words, S/KW :	Distributed systems, real-time collaboration, CRDT, RGA, microservices, WebSocket	
UC		
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad	
Note, N :		
Abstract, AB :	This thesis presents the design and implementation of CodeStream, a distributed system for real-time collaborative source code editing. Conflicts caused by simultaneous edits are resolved using a CRDT structure based on the RGA algorithm. The system consists of independent Rust microservices communicating synchronously (HTTP) and asynchronously (RabbitMQ), with JWT authentication, Redis-based document state synchronization, and isolated code execution inside Docker containers.	
Accepted by the Scientific Board on, ASB :		
Defended on, DE :	01.01.2025	
Defended Board, DB :	President:	Petar Petrović, Phd., assoc. professor
	Member:	Marko Marković, Phd., asist. professor
	Member:	
	Member, Mentor:	Igor Dejanović, Phd., full professor
		Menthor's sign



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

Садржај

1	Увод	1
1.1	Предмет и циљ рада	1
1.2	Структура рада	2
2	Стање у области	3
2.1	Проблем истовремених измена	3
2.2	Операциона трансформација	3
2.3	CRDT приступ	4
2.4	Преглед постојећих алата	5
3	Архитектура система	7
3.1	Општи преглед архитектуре	7
3.2	Комуникација између сервиса	9
3.3	Технолошки стек	10
3.4	Контејнеризација и покретање система	11
4	CRDT алгоритам: пројектовање и имплементација	13
4.1	Структура података RGA документа	13
4.2	Лампортов логички сат	14
4.3	Уметање карактера	15
4.4	Брисање карактера и tombstone механизам	16
4.5	Ток обраде операције кроз систем	17
4.6	Чување и учитавање стања документа	18
5	Имплементација сервиса и клијентске апликације	19
5.1	Auth & User Service	19
5.2	Session Gateway Service	21
5.3	Code Execution Service	23
5.4	Клијентска апликација	24
6	Тестирање и евалуација система	27
6.1	Приступ тестирању	27
6.2	Функционални сценарији	27
6.3	Безбедносна разматрања	29
6.4	Ограничења приступа тестирању	29
7	Закључак	31
7.1	Осврт на остварене резултате	31
7.2	Ограничења решења	31
7.3	Правци даљег развоја	32
	Списак слика	35

Списак листинга	37
Списак табела	39
Списак коришћених скраћеница	41
Списак коришћених појмова	43
Биографија	45
Литература	47

Колаборативно уређивање садржаја у реалном времену постало је уобичајен начин рада у савременим софтверским алатима. Апликације попут алата Google Docs [1], Figma-e [2] и алата Visual Studio Code Live Share [3] омогућавају да више корисника истовремено гледа и мења исти документ, при чему се измене сваког корисника тренутно приказују осталим учесницима сесије. Овакав начин рада данас се очекује и приликом заједничког писања и извршавања програмског кода, нарочито у контексту наставе програмирања, техничких интервјуа и удаљеног (енг. *remote*) тимског рада, где више особа треба да ради на истој датотеци без потребе за ручним слањем измена или наизменичним преузимањем контроле.

Проблем који при томе настаје јесте усклађивање истовремених измена које потичу од различитих корисника над истим делом документа. Наивна решења, попут закључавања документа за једног корисника или правила „последња измена побеђује“ (енг. *last write wins*), доводе до губитка измена или до онемогућавања праве истовремене сарадње. Због тога су у пракси развијена два основна приступа усклађивању истовремених измена над дељеним текстуалним садржајем: операциона трансформација (енг. *Operational Transformation*, OT) [4] и структуре без конфликта за реплицирано усклађивање података (енг. *Conflict-free Replicated Data Types*, CRDT) [5]. Док операциона трансформација захтева централизоване сервер који трансформише и уређује редослед операција, CRDT приступ гарантује да ће све реплике документа конвергирати ка истом стању без обзира на редослед операција, што поједностављује архитектуру дистрибуираног система и отвара могућност за рад и без сталне везе са сервером.

1.1 Предмет и циљ рада

Предмет овог рада јесте пројектовање и имплементација дистрибуираног система за колаборативно уређивање програмског кода у реалном времену (у даљем тексту CodeStream). Циљ рада је да се практично примени CRDT приступ усклађивању истовремених измена, конкретно RGA (енг. *Replicated Growable Array*) алгоритам [6], у контексту система који, поред саме синхронизације текста, омогућава управљање корисницима и пројектима, аутентификацију и контролу права приступа, као и безбедно извршавање кода који корисници заједно напишу.

Систем је реализован као скуп независних микросервиса писаних у програмском језику Rust, који комуницирају међусобно синхроно (HTTP) и асинхроно (RabbitMQ). У раду се посебна пажња посвећује:

- анализи постојећих приступа и алата за колаборативно уређивање садржаја у реалном времену,
- пројектовању и имплементацији CRDT структуре података и алгорита за разрешавање конфликта,
- пројектовању микросервисне архитектуре и комуникације између сервиса, и
- имплементацији подсистема за изоловано извршавање кода унутар Docker контејнера.

1.2 Структура рада

Рад је организован у седам поглавља. Поглавље 2 даје преглед стања у области колаборативног уређивања садржаја у реалном времену, упоређује операциону трансформацију и CRDT приступ и приказује неколико CRDT структура намењених уређивању секвенци (низова карактера). Поглавље 3 описује општу архитектуру система, коришћене технологије и начин комуникације између сервиса. Поглавље 4 детаљно приказује пројектовање и имплементацију RGA алгорита коришћеног у оквиру CRDT сервиса за синхронизацију. Поглавље 5 приказује имплементацију осталих сервиса система (аутентификација и управљање корисницима, посредовање у реалном времену, извршавање кода) и веб апликације на страни клијента. Поглавље 6 приказује начин тестирања и евалуације система. Коначно, поглавље 7 сумира резултате рада, идентификује ограничења тренутног решења и предлаже правце даљег развоја система.

Глава 2

Стање у области

2.1 Проблем истовремених измена

Када више корисника независно уређује исти документ, неопходно је одлучити шта се дешава када два корисника, у истом тренутку (или пре него што сазнају за туђу измену), измене исти или суседни део текста. Уколико би се измене примењивале наивно, редоследом којим стижу на сервер, дошло би до проблема померања позиције (енг. *positional indexing problem*). Уколико се измена корисника А односи на позицију 10 у документу, а измена корисника Б, извршена нешто раније, већ је уметнула пет нових карактера пре позиције 10, дословна примена измене корисника А на позицију 10 у измењеном документу довела би до погрешног резултата. Ово важи чак и када корисници уопште не мењају исти део текста. Довољно је да измене утичу на релативне позиције карактера након њих.

У литератури постоје два основна приступа решавању овог проблема: операциона трансформација и CRDT структуре података.

2.2 Операциона трансформација

Операциона трансформација (ОТ) је приступ представљен крајем осамдесетих година прошлог века [4], а касније усавршен и примењен у више комерцијалних система, међу којима је и Google Docs [1]. Основна идеја ОТ приступа јесте да се, пре примене операције која је настала на застарелој верзији документа, та операција трансформише у односу на операције које су у међувремену већ примењене, тако да крајњи ефекат остане исправан без обзира на редослед приспећа. На пример, уколико је операција уметања намеравала да дода карактер на позицију 10, а у међувремену је пре те позиције убачено пет нових карактера, трансформисана операција ће уместо тога циљати позицију 15. ОТ овим поступком успешно решава проблем померања позиције.

Проблем ОТ приступа је што трансформационе функције морају задовољити строге математичке услове како би резултат био исправан независно од редоследа примене трансформација, а Остер и др. [7] показују да доказивање исправности ових функција за сложеније структуре података у пракси често доводи до грешака у имплементацији. Због тога се ОТ системи по правилу ослањају на централизовани сервер који одржава редослед операција и врши трансформације. Клијенти

међусобно не морају да се директно усклађују, али систем захтева сталну везу са тим сервером.

2.3 CRDT приступ

CRDT структуре података представљају алтернативни приступ у коме се проблем усклађивања решава самом структуром података, а не посебним алгоритмом трансформације [5]. CRDT структура гарантује да ће све реплике, након што приме исти скуп операција (без обзира на редослед и без обзира на то да ли се нека операција примени више од једном), доћи до истог коначног стања. Ово својство је познато као **јака коначна конзистентност** (енг. *strong eventual consistency*).

Према раду [5], разликују се две врсте CRDT структура:

- **CRDT засноване на стању** (енг. *state-based*) – реплике периодично размењују своје потпуно стање,
- **CRDT засноване на операцијама** (енг. *operation-based*) – реплике размењују само операције (нпр. „уметни карактер X после карактера Y“).

CRDT Sync Service у склопу имплементације користи модел ближи операционо-заснованом приступу. Преноси се конкретна операција уметања или брисања карактера (поглавље 4), а не цело стање документа при свакој измени.

2.3.1 Секвенцијалне CRDT структуре

Посебан изазов представља примена CRDT приступа на секвенце (низове карактера), пошто позиција елемента зависи од свих претходних елемената, а не само од његовог непосредног претходника. Наивна решења често пате од проблема интерливинга (енг. *interleaving anomaly*) – када два корисника истовремено уносе по низ карактера на исту позицију, резултујући текст може испасти испреплетан (нпр. уместо „abcdef“ добије се „adbef“). Развијено је више структура које решавају овај проблем на различите начине:

- **WOOT** [7] – сваки карактер носи идентификатор и референце на свог претходника и следбеника у тренутку уметања; уметање новог карактера захтева претрагу и поређење са свим карактерима између претходника и следбеника, што у најгорем случају доводи до квадратне сложености у односу на број карактера.
- **RGA** [6] – сваки карактер носи јединствени идентификатор заснован на логичком (Лампортовом) сату [8] и идентификатору реплике; конфликти при уметању на исту позицију решавају се поређењем идентификатора. Овај алгоритам је описан детаљно у поглављу 4, пошто представља основу CRDT Sync Service-a.
- **LSEQ** [9] – гради идентификаторе позиција у стаблу променљиве ширине (енг. *exponential tree*) тако да се избегне непрестано растуће дужина идентификатора

карактера код структура сличних Treedoc-у, уз циљ равномерне расподеле идентификатора и смањења вероватноће колизија при истовременом уметању на исту позицију.

- **YATA** – алгоритам коришћен у оквиру система Yjs [10], [11], сличан RGA-у по идеји уређења суседа по идентификатору, али оптимизован тако да се суседни карактери исте реплике групишу у „блокове“ (енг. *splits/items*), чиме се смањује број појединачних објеката које треба одржавати у меморији у односу на структуру која сваки карактер чува као посебан елемент низа, попут тренутне имплементације RGA у систему CodeStream.
- **Automerge / CRDT над JSON структурама** [12], [13] – уместо да се ограничи на секвенце карактера, овај приступ уопштава CRDT принципе на произвољне угнежђене JSON структуре (листе, мапе, текст), што омогућава колаборативно уређивање сложенијих докумената, не само равног текста.

Тренутна имплементација у систему CodeStream (поглавље 4) најближа је основном RGA приступу – сваки карактер је засебан елемент низа, без груписања у блокове попут YATA-е, што поједностављује имплементацију по цену линеарне сложености уметања.

2.4 Преглед постојећих алата

Табела 1 даје кратак преглед приступа које користи неколико познатих алата за колаборативно уређивање садржаја у реалном времену.

Табела 1: Приступи усклађивању истовремених измена код изабраних алата за колаборативно уређивање.

Алат	Приступ	Напомена
Google Docs [1]	Операциона трансформација	Централизовани сервер одржава ауторитативни редослед операција и трансформише операције клијената пре примене.
Figma [2]	Прилагођени, CRDT-инспирисани систем	Користи властити бинарни протокол и структуре сличне CRDT приступу, оптимизоване за објекте векторског цртежа, а не за линеарни текст.

Алат	Приступ	Напомена
VS Code Live Share [3]	Модел домаћина (<i>host</i>)	Измене госта (<i>guest</i>) се пре- носе ка ауторитативној ин- станци уређивачког окру- жења домаћина; не ослања се на CRDT или ОТ спајање независних реплика.
CodeStream	CRDT (RGA)	Централизовани сервис за синхронизацију примењу- је RGA алгоритам над опе- рацијама које стижу од свих корисника (поглавље 4).

Треба уочити да ниједан од наведених алата, укључујући и CodeStream, не пред- ставља „чист“ пример потпуно децентрализованог CRDT система без централног ауторитета. Сви и даље у пракси пролазе кроз сервер који посредује у комуникаци- ји, из практичних разлога (нпр. потреба за трајним чувањем документа, контролом приступа, обавештавањем корисника који нису тренутно на вези). Предност CRDT приступа у оваквим системима није нужно у потпуном уклањању сервера, већ у једноставнијем и доказиво исправном моделу спајања измена, као и у могућности да се, уз додатне измене, систем прошири ка подршци за рад без сталне везе са сервером (*offline* режим), што ОТ приступ отежава услед своје зависности од редо- следа примене трансформација.

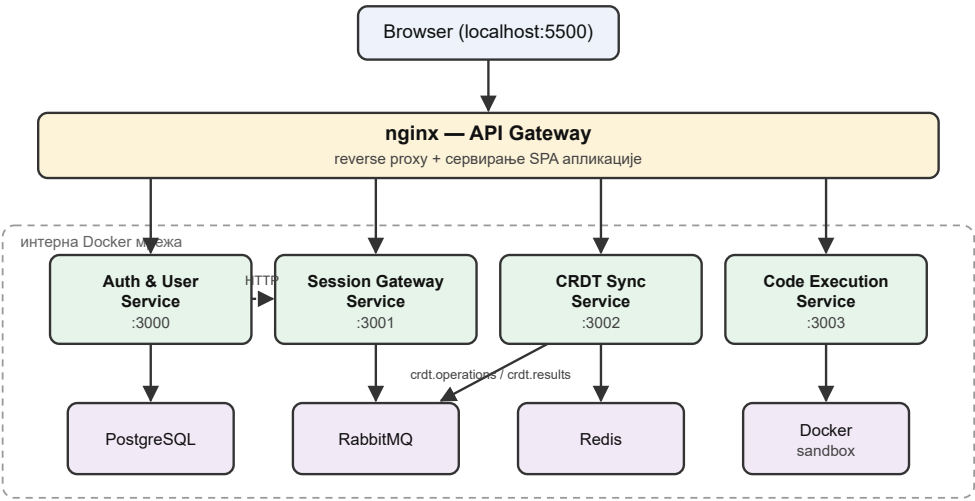
Глава 3

Архитектура система

CodeStream је реализован као микросервисна апликација, односно скуп независних процеса који се независно граде, покрећу и скалирају, а међусобно комуницирају путем мреже [14]. Овакав приступ пројектовању одабран је како би се јасно раздвојиле надлежности система: управљање идентитетом корисника, посредовање у комуникацији у реалном времену, разрешавање конфликта над дељеним документом и извршавање кода.

3.1 Општи преглед архитектуре

Систем се састоји од четири сервиса писана у програмском језику Rust, релационе базе података, кеш базе, система за размену порука и *nginx reverse proxy* сервера који представља јединствену улазну тачку за сав саобраћај из веб прегледача. Слика 1 приказује компоненте система и њихову међусобну комуникацију.



Слика 1: Архитектура система CodeStream.

Табела 2 сумира одговорности и спољашње зависности сваког од четири сервиса.

Табела 2: Преглед сервиса система CodeStream.

Сервис	Порт	Одговорност	Спољашња зави- сност
Auth & User Service	3000	Регистрација и при- јава корисника, из- давање и валидација JWT, CRUD операције над пројектима, кон- трола права приступа (owner / write / read)	PostgreSQL
Session Gateway Service	3001	Прихватање WebSocket конекци- ја, провера приступа пројекту, праћење по- зиција курсора и про- мена језика, објављи- вање и читање надола- зећих CRDT операција	RabbitMQ, Auth service (HTTP)
CRDT Sync Service	3002	Примена RGA алго- ритма над операци- јама уметања/брисања карактера, чување и достављање стања до- кумента	Redis, RabbitMQ
Code Execution Service	3003	Извршавање кода ко- рисника унутар изо- лованог Docker кон- тејнера	Docker (преко Unix socket-a)

Сви сервиси деле заједничку библиотеку shared, у оквиру које се налазе: Auth extractor AuthUser који аутоматски валидира JWT из Authorization заглавља, структура Claims која одговара садржају JWT (sub, email, exp), као и типови порука CrdtOperationMessage и CrdtResultMessage који се размењују преко RabbitMQ-a. Овим се избегава дуплирање логике за аутентификацију и дефиниције порука у сваком сервису понаособ.

3.2 Комуникација између сервиса

Сервиси комуницирају на два начина: синхронно, путем HTTP протокола, и асинхронно, путем система за размену порука RabbitMQ.

Синхрона комуникација. Session Gateway Service, приликом успостављања сваке WebSocket конекције, локално проверава важност JWT токена, а затим синхроним HTTP захтевом (GET /projects/:id) пита Auth & User Service да ли корисник има право приступа траженом пројекту. Овај позив се извршава само једном по конекцији, приликом успостављања везе, а добијена улога (owner, write или read) одређује на који начин ће Session Gateway Service обрађивати све накнадне поруке од тог корисника.

Асинхрона комуникација. Измене над документом се преносе кроз два реда порука на RabbitMQ-у:

- `crdt.operations` – Session Gateway Service објављује операције уметања/брисања карактера које пристигну од корисника (fire-and-forget, без чекања на одговор), а CRDT Sync Service их конзумира.
- `crdt.results` – CRDT Sync Service, након примене RGA алгоритма, публикује решену операцију заједно са ажурираним текстом документа, а Session Gateway Service то прихвата у засебном Tokio *task*-у и јавља свим активним конекцијама у одговарајућој „соби“ (пројекту).

Овакав дизајн раздваја пријем корисничких измена од њихове обраде. Session Gateway Service никада не чека CRDT Sync Service да заврши обраду пре него што прихвати следећу поруку од клијента, чиме се смањује утицај евентуалног успоравања CRDT сервиса. Цена оваквог приступа је што су оба реда декларисана као уобичајени (енг. *non-durable*) редови, па се у случају пада RabbitMQ процеса поруке које нису још обрађене губе.

Комуникација између веб прегледача и система у потпуности пролази кроз nginx, који игра улогу *reverse proxy* сервера и истовремено сервира статичке фајлове *frontend* апликације (енг. *Single Page Application*, скраћено SPA). Табела 3 приказује мапирање рута на сервисе.

Табела 3: Мапирање HTTP рута на сервисе у оквиру nginx конфигурације.

Рута	Циљни сервис
/api/auth/*	Auth & User Service (:3000)
/api/crdt/*	CRDT Sync Service (:3002)
/api/exec/*	Code Execution Service (:3003)
/ws/*	Session Gateway Service (:3001), WebSocket упржејд
остало (/*)	статички фајлови <i>frontend</i> апликације (SPA fallback)

3.3 Технолошки стек

Табела 4 приказује преглед кључних технологија и библиотека коришћених у оквиру система, заједно са верзијама коришћеним у тренутку писања рада.

Табела 4: Преглед технолошког стека система CodeStream.

Компонента	Технологија	Верзија	Улога
Backend	Rust	-	Програмски језик свих сервиса
Backend	axum	0.7.9	Веб оквир за HTTP и WebSocket руте
Backend	tokio	1.50.0	Асинхрони <i>runtime</i>
Backend	sqlx	0.7.4	Приступ PostgreSQL бази и миграције
Backend	lapin	2.5.5	AMQP клијент за RabbitMQ
Backend	bollard	0.17.1	Управљање Docker контејнерима
Backend	redis	0.27.6	Клијент за Redis базу
Backend	jsonwebtoken	9.3.1	Издавање и валидација JWT токена
Backend	argon2	0.5.3	Хеширање лозинки
Backend	serde / serde_json	1.0.228	(Де)серијализација порука и одговора
Backend	uuid	1.23.1	Јединствени идентификатори

Компонента	Технологија	Верзија	Улога
Backend	request	0.12.28	HTTP клијент (Gateway → Auth сервис)
Frontend	React	18.3.1	Библиотека за кориснички интерфејс
Frontend	Vite	6.0.11	<i>Development server</i> и <i>build</i> алат
Frontend	@uiw/react-codemirror	4.23.0	Едитор кода (CodeMirror 6)
Инфраструктура	PostgreSQL	16	Релациона база (корисници, пројекти, права приступа)
Инфраструктура	Redis	7	Складиштење тренутног стања CRDT документа
Инфраструктура	RabbitMQ	3	Размена порука између сервиса
Инфраструктура	nginx	alpine	API gateway / reverse proxy / SPA сервер
Инфраструктура	Docker / Docker Compose	-	Контејнеризација и оркестрација

3.4 Контејнеризација и покретање система

Цео систем се покреће помоћу Docker Compose-а. Свака инфраструктурна компонента (PostgreSQL, Redis, RabbitMQ) има дефинисану проверу здравља (енг. *healthcheck*), а зависни сервиси користе услов `condition: service_healthy` како би сачекали да инфраструктура буде спремна пре него што покушају да јој приступе.

Сва четири Rust сервиса се граде помоћу истог обрасца двофазног (енг. *multi-stage*) Docker build-а: у првој фази (`rust:slim`) се компајлира само бинарни фајл конкретног сервиса, док се у другој фази (`debian:bookworm-slim`) преузима искључиво компајлирани бинарни фајл и минималне системске зависности. На овај начин се крајња слика значајно смањује у односу на слику која би садржала цео Rust *toolchain*. Треба напоменути да свака од Dockerfile дефиниција копира изворни код свих сервиса у контекст изградње, иако компајлира само један бинарни фајл. Последица тога је да измена изворног кода било ког сервиса поништава Docker кеш

слојеве за изградњу свих осталих сервиса, што продужава време изградње целог система приликом развоја.

Frontend се гради на сличан начин: `node:20-alpine` фаза извршава `npm run build`, а резултујући статички фајлови се копирају у `nginx:alpine runtime` слику заједно са конфигурацијом *reverse proxy*-ја описаном у табели 3.

Сервис за извршавање кода (Code Execution Service) представља изузетак у односу на остале сервисе по питању контејнеризације: уместо да покреће сопствене контејнере „унутар себе“, он преко монтираног Unix socket-а (`/var/run/docker.sock`) управља Docker демоном хост машине и на њему покреће привремене контејнере за извршавање корисничког кода (тзв. *Docker-outside-of-Docker* образац). Ово поједностављује имплементацију, али значи да компромитовање овог сервиса даје потпуну контролу над Docker демоном хоста, што представља битно безбедносно разматрање приликом евентуалног постављања система у производно окружење.

Глава 4

CRDT алгоритам: пројектовање и имплементација

У овом поглављу је детаљно приказана имплементација RGA алгоритма [6] у оквиру CRDT Sync Service-a, укључујући структуре података, алгоритам уметања и брисања карактера, ток обраде операције кроз систем, као и начин чувања стања документа. Изворни код приказан у овом поглављу потиче из фајла `services/crdt-sync-service/src/rga.rs`.

4.1 Структура података RGA документа

Основна идеја RGA структуре је да се документ представља као секвенца карактера код које сваки карактер има јединствен, глобално упоредив идентификатор, независно од своје тренутне позиције у тексту. Идентификатор карактера (`CharId`) чине два поља: логички сат (`clock`) и идентификатор корисника (`user_id`) који је извршио уметање (листинг 1).

```
#[derive(Serialize, Deserialize, Clone, Debug, PartialEq, Eq)]
pub struct CharId {
    pub clock: u64,
    pub user_id: Uuid,
}

impl Ord for CharId {
    fn cmp(&self, other: &Self) -> std::cmp::Ordering {
        self.clock
            .cmp(&other.clock)
            .then_with(|| self.user_id.cmp(&other.user_id))
    }
}
```

Листинг 1: Идентификатор карактера `CharId` и релација уређења над њим.

Релација уређења (`Ord`) над `CharId` вредностима је лексикографска: карактер са већим `clock` пољем се сматра „већи“, а уколико су два карактера добила исти логички сат, што се, како је објашњено у одељку 4.2, у тренутној имплементацији не може десити – предност има карактер чији је `user_id` већи. Овим је обезбеђено да је релација уређења тотална и детерминистичка: за било која два карактера у систему увек постоји јасан одговор који је „већи“, независно од тога на ком рачунару или у ком тренутку је поређење извршено. Ово својство је кључно за конвергенцију

CRDT структура. Сви учесници коџи примене исти скуп операција, без обзира на редослед приспеџа, морају доџи до истог коначног стаџа документа [5].

Сам карактер у документу (`RgaChar`) чува садржај, идентификатор, идентификатор претходног карактера у односу на коџи је уметнут (`prev_id`) и заставицу `deleted` која означава да ли је карактер логички обрисан. Документ (`RgaDocument`) се представља као уреџени низ (`Vec<RgaChar>`) заједно са бројачем логичког сата (листинг 2).

```
pub struct RgaChar {
    pub id: CharId,
    pub content: char,
    pub deleted: bool,
    pub prev_id: Option<CharId>,
}

pub struct RgaDocument {
    pub chars: Vec<RgaChar>,
    pub clock: u64,
}

#[serde(tag = "type", rename_all = "snake_case")]
pub enum ClientOperation {
    Insert { after_char_id: Option<CharId>, content: char },
    Delete { char_id: CharId },
}
```

Листинг 2: Структуре `RgaChar`, `RgaDocument` и операције које стижу од клијента.

Треба уочити да се документ не представља ни као повезана листа (са показивачима на следеџи елемент) ни као стабло, веџ као обичан низ (`Vec`) коџи се одржава у исправном редоследу приликом сваког уметаџа. Ово поједностављује имплементацију, али доводи до тога да је сложеност уметаџа линеарна у односу на дужину документа (детаљније у одељку 4.3), за разлику од структура попут YATA [10] или RGA-Split структуре засноване на блоковима карактера, које постижу боље перформансе код великог броја истовремених измена (видети и одељак 2).

4.2 Лампортов логички сат

Свака операција уметаџа добија нови идентификатор чији се `clock` добија инкрементирањем бројача `RgaDocument.clock`. За разлику од класичног Лампортовог логичког сата [8], коџи се независно одржава на свакој реплици и усклађује при пријему туџих порука узимањем максимума, у тренутној имплементацији бројач се одржава централизовано, унутар CRDT Sync Service-а. Сваки документ има тачно једну активну инстанцу `RgaDocument` чије стаџе се учитава из, и уписује у, Redis базу. Због тога, у оквиру једног документа, две операције уметаџа никада не могу

добити исти clock, пошто их обрађује један те исти процес, редом којим стижу из RabbitMQ реда crdt.operations.

Ова одлука значајно поједностављује имплементацију у односу на потпуно децентрализован CRDT систем у ком би сваки клијент самостално додељивао идентификаторе новим карактерима, а сервер би морао да усклађује сатове при пријему операција са других реплика. Цена оваквог приступа је да CRDT Sync Service постаје једина тачка која додељује редослед операцијама над конкретним документом, што представља уско грло.

4.3 Уметање карактера

Приликом уметања новог карактера, алгоритам прво проналази позицију претходног карактера (prev_id) у низу, а затим помера позицију уметања удесно преко свих суседних карактера (карактера са истим prev_id) чији је идентификатор већи од идентификатора новог карактера (листинг 3). На овај начин се, међу карактерима уметнутим на исту позицију, успоставља редослед по опадајућем идентификатору. Карактер са већим идентификатором увек завршава ближе заједничком претходнику, независно од тога којим редоследом су операције стигле на сервер.

```
fn insert_char(
    &mut self,
    id: CharId,
    content: char,
    prev_id: Option<CharId>,
) -> Result<(), String> {
    let start_idx = match &prev_id {
        None => 0,
        Some(pid) => self.chars.iter()
            .position(|c| &c.id == pid)
            .ok_or("Predecessor character not found")? + 1,
    };

    let insert_at = self.chars[start_idx..]
        .iter().enumerate()
        .take_while(|(_, c)| c.prev_id == prev_id && c.id > id)
        .map(|(i, _)| start_idx + i + 1)
        .last()
        .unwrap_or(start_idx);

    self.chars.insert(insert_at, RgaChar { id, content, deleted: false,
        prev_id });
    Ok(())
}
```

Листинг 3: Алгоритам уметања карактера у RGA документ.

Сложеност овог поступка је $O(n)$ у односу на укупан број карактера у документу (укључујући и оне обрисане, о чему више у одељку 4.4), пошто се и претрага претходног карактера и померање елемената унутар Vec структуре извршавају линеарно. За документе умерене величине (нпр. изворни код појединачне датотеке) ово не представља проблем у пракси, али за веома дуге сесије уређивања ово поставља горњу границу перформанси система (видети поглавље 7).

4.3.1 Пример разрешавања конфликта

Табела 5 илуструје понашање алгорита када два корисника готово истовремено уметну по један карактер непосредно након истог постојећег карактера X. Пошто CRDT Sync Service обрађује операције секвенцијално, једна од њих ће увек бити обрађена прва. У овом примеру прва је обрађена операција корисника A.

Табела 5: Разрешавање конфликта код два уметања на истој позицији.

Корак	Операција	Стање документа после операције
1	Почетно стање	X
2	Корисник A уметне a после X – добија id = (5, UUID-A)	X, a(5,A)
3	Корисник B уметне b после X – добија id = (6, UUID-B)	X, b(6,B), a(5,A)

Поред тога што је операција корисника B обрађена **после** операције корисника A, њен карактер b завршава **испред** карактера a у коначном тексту, зато што има већи идентификатор ($clock = 6 > 5$). Редослед међу карактерима уметнутим на исту позицију не зависи од редоследа обраде операција, већ искључиво од поређења њихових идентификатора, чиме је загарантована конвергенција: без обзира на то којим редоследом би, у децентрализованом сценарију, различите реплике примале ове две операције, обе би дошле до истог коначног резултата X, b, a.

4.4 Брисање карактера и tombstone механизам

Брисање карактера не уклања елемент из низа, већ само поставља заставицу deleted на true (листинг 4). Оваква ознака за обрисан карактер (енг. *tombstone*) мора остати сачувана зато што касније приспеле операције уметања могу референцирати управо тај (обрисани) карактер преко поља rev_id. Уколико би обрисани карактер био потпуно уклоњен из низа, таква операција уметања не би могла да пронађе своју позицију.

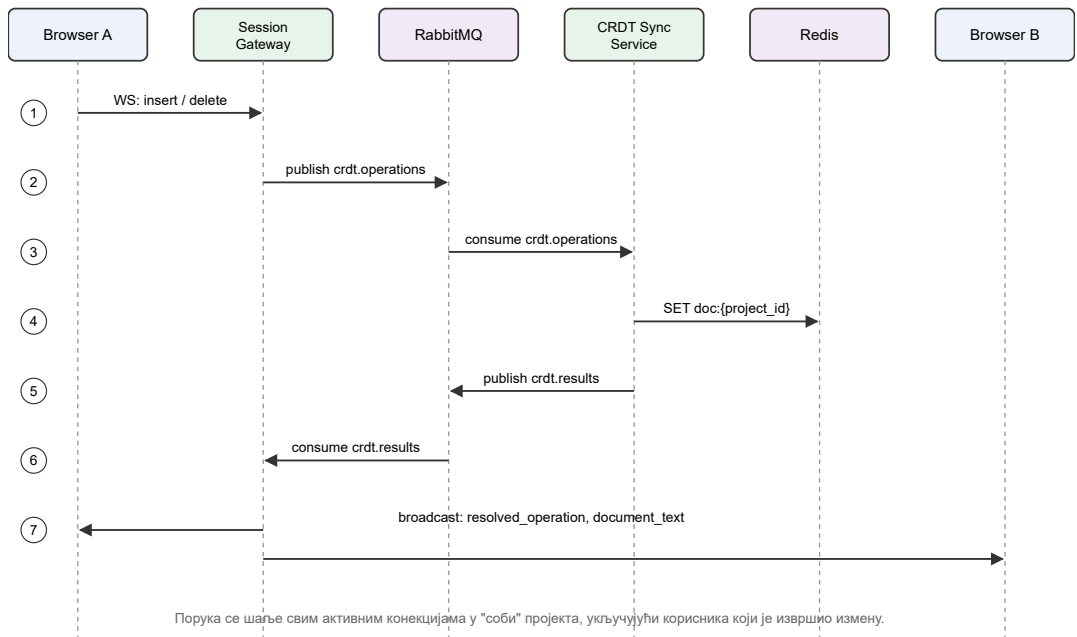
```
fn delete_char(&mut self, char_id: &CharId) -> Result<(), String> {
    self.chars.iter_mut()
        .find(|c| &c.id == char_id)
        .ok_or_else(|| "Character not found".to_string())
        .map(|c| c.deleted = true)
}
```

Листинг 4: Означавање карактера као обрисаног (*tombstone*).

Приликом генерисања текста документа за приказ кориснику, сви карактери означени као обрисани се једноставно изостављају. Проблем оваквог приступа је што се *tombstone* карактери никада не уклањају из низа нити из серијализованог стања у Redis бази. Документ у меморији и даље расте са сваким обрисаним карактером, чак и када он више никада неће бити видљив кориснику. Ово представља једно од главних ограничења тренутне имплементације.

4.5 Ток обраде операције кроз систем

Слика 2 приказује потпун ток једне операције уметања или брисања, од тренутка када је корисник унесе у прегледачу, до тренутка када је резултат приказан свим осталим учесницима сесије.



Слика 2: Ток обраде CRDT операције кроз систем.

Session Gateway Service публикује операцију на ред `crdt.operations` одмах по пријему, не чекајући да CRDT Sync Service заврши обраду (*fire-and-forget*). Одзивност

зависи само од брзине којом ће резултат стићи назад преко реда `crdt.results` и бити *broadcast*-ован свим конекцијама у „соби“ пројекта, укључујући и корисника који је извршио измену.

4.6 Чување и учитавање стања документа

После сваке успешно примењене операције, цео `RgaDocument` (заједно са свим *tombstone* карактерима) се серијализује у JSON формат и уписује у Redis под кључем облика `doc:{id_пројекта}` (обичном SET наредбом, без TTL-а и без чувања историје претходних стања). Пошто систем нема посебан ентитет „документ“ одвојен од пројекта, идентификатор пројекта се уједно користи и као идентификатор документа.

Приликом отварања едитора, клијент прво позива `GET /documents/:id/state` како би прочитао тренутно стање документа пре него што успостави WebSocket конекцију. Овај захтев захтева важећи JWT токен, али, за разлику од осталих *endpoint*-а који раде над конкретним пројектом, не проверава да ли корисник заиста има одобрен приступ том пројекту, нити из одговора изоставља *tombstone* карактере. Ово представља пропуст у контроли приступа који је детаљније коментарисан у поглављу [6](#).

Глава 5

Имплементација сервиса и клијентске апликације

Ово поглавље приказује имплементацију преостала три сервиса, Auth & User Service, Session Gateway Service и Code Execution Service, као и клијентске апликације. За разлику од CRDT Sync Service-а, приказаног у поглављу 4, ови сервиси не примењују CRDT алгоритам, али обезбеђују идентитет, права приступа, посредовање у комуникацији у реалном времену и извршавање кода без којих CRDT механизам не би био употребљив у оквиру целовитог система.

5.1 Auth & User Service

Auth & User Service је једини сервис који директно приступа релационој бази података. Задужен је за регистрацију и пријаву корисника, издавање и валидацију JWT-а и управљање пројектима и правима приступа над њима.

5.1.1 Шема базе података

Шема се примењује аутоматски при покретању сервиса помоћу `sqlx::migrate!` механизма. Садржи три табеле: `users`, `projects` и `project_access` (листинг 5).

```
CREATE TABLE users (  
    id UUID PRIMARY KEY,  
    email VARCHAR(255) UNIQUE NOT NULL,  
    password_hash VARCHAR(255) NOT NULL,  
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
CREATE TABLE projects (  
    id UUID PRIMARY KEY,  
    name VARCHAR(255) NOT NULL,  
    owner_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
CREATE TABLE project_access (  
    project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,  
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
    role VARCHAR(10) NOT NULL CHECK (role IN ('read', 'write')),  
    PRIMARY KEY (project_id, user_id)  
);
```

Листинг 5: Шема релационе базе података – корисници, пројекти и права приступа.

Улога `owner` се не чува као посебан ред у табели `project_access` већ се изводи из поља `projects.owner_id`. Корисник има улогу власника уколико је управо он креирао пројекат. Уколико то није случај, улога се тражи у табели `project_access`, а ако ни тамо не постоји одговарајући ред, приступ се одбија са статусом 403 Forbidden. Само власник пројекта може да додели приступ другом кориснику (`POST /projects/:id/access`) или да обрише пројекат; систем тренутно не нуди могућност опозива већ додељеног приступа.

5.1.2 Аутентификација

Лозинке се хешују помоћу Argon2id алгоритма библиотеке `argon2`. У коду нису подешени прилагођени параметри, што је вредно напоменути с обзиром на то да се препоручене вредности параметара трошка обично прилагођавају очекиваном хардверу сервера [15].

Приликом пријаве, унета лозинка се проверава у односу на РНС низ сачуван у бази података, који садржи хеш лозинке и параметре коришћене за његово израчунавање (листинг 6).

```
let salt = SaltString::generate(&mut OsRng);
let argon2 = Argon2::default();
let password_hash = argon2
    .hash_password(payload.password.as_bytes(), &salt)?
    .to_string();
```

Листинг 6: Хеширање лозинке приликом регистрације корисника.

По успешној пријави издаје се JWT токен потписан HS256 алгоритмом, важећи 24 часа. Садржај токена (Claims) дефинисан је у заједничкој `shared` библиотеци и садржи идентификатор корисника, имејл адресу и време истека (листинг 7), тако да сваки сервис може самостално да провери важност токена без додатног позива ка Auth & User Service-у.

```
pub struct Claims {
    pub sub: Uuid,
    pub email: String,
    pub exp: usize,
}
```

Листинг 7: Садржај JWT токена (Claims).

Ова провера, реализована кроз `AuthUser Axum extractor` у `shared` сандуку, чита `JWT_SECRET` променљиву окружења при свакој обради захтева и верификује токен HS256 алгоритмом коришћењем истог симетричног кључа на свим сервисима.

5.1.3 Преглед HTTP рута

Табела 6 приказује списак HTTP рута сервиса за управљање корисницима. Такође приказује да ли приступ наведеним рутама захтева JWT.

Табела 6: HTTP руте сервиса Auth & User Service.

Метода	Рута	Опис	Захтева JWT
GET	/health	Провера доступности сервиса	не
POST	/register	Регистрација новог корисника	не
POST	/login	Пријава, издавање JWT токена	не
GET	/me	Подаци о пријављеном кориснику	да
GET	/projects	Листа пројеката корисника	да
POST	/projects	Креирање новог пројекта	да
GET	/projects/:id	Детаљи о пројекту (користи и Session Gateway)	да
DELETE	/projects/:id	Брисање пројекта (само власник)	да
POST	/projects/:id/access	Додела приступа другом кориснику (само власник)	да

5.2 Session Gateway Service

Session Gateway Service представља једину real-time везу између прегледача и остатка система. Стање сервиса, укључујући мапу активних „соба“ по пројекту, дефинисано је структуром AppState (листинг 8).

```
pub struct AppState {
    pub rooms: Arc<RwLock<HashMap<Uuid, broadcast::Sender<String>>>>,
    pub jwt_secret: String,
    pub auth_service_url: String,
    pub http_client: reqwest::Client,
    pub rabbit_channel: lapin::Channel,
}
```

Листинг 8: Дељено стање AppState сервиса Session Gateway.

Свака „соба“ (пројекат) добија сопствени *broadcast* канал капацитета 256 порука. Уколико неки клијент заостане у читању (спор конзумент), прима грешку *lagged* и наставља пријем следећих порука без покушаја надокнаде пропуштених. Ово не нарушава исправност приказа документа, јер свака порука која представља резултат CRDT операције садржи целокупан текст документа у тренутном стању. Стога, уколико клијент пропусти једну или више порука због кашњења, пријемом наредне поруке добија актуелно стање документа и може да настави са приказом без потребе за накнадним преузимањем пропуштених порука.

5.2.1 Успостављање везе и провера приступа

Приликом успостављања WebSocket везе, клијент шаље JWT токен као упитни параметар (`?token=...`) уз URL. Сервис прво локално верификује потпис и важност токена, а затим синхроним HTTP позивом проверава да ли корисник има одобрен приступ траженом пројекту и коју улогу има (`owner`, `write` или `read`). Само корисници са улогом `owner` или `write` могу да шаљу измене над документом. Покушаји уређивања од стране корисника са улогом `read` се тихо одбацују, без повратне поруке о грешци.

5.2.2 Обрада порука

За разлику од CRDT Sync Service-a, који користи строго типизираних Rust enum структуре (`ClientOperation`, `ResolvedOperation`, поглавље 4), Session Gateway Service поруке које стижу од клијента парсира као неструктурирани JSON (`serde_json::Value`) и рутира их на основу текстуалног поља `"type"`:

- `"cursor"` – позиција курсора се проследи осталим учесницима собе, без чувања и без пролаза кроз RabbitMQ,
- `"language"` – промена програмског језика се проследи осталим учесницима,
- `"disconnect"` – сигнал да је корисник намерно напустио сесију,
- било који други тип, уколико корисник има права уписа – умотава се у `CrdtOperationMessage` и публикује на ред `crdt.operations`.

Поруке које сервис шаље ка клијентима су обликоване помоћу јединствене омотнице `BroadcastEnvelope` (листинг 9), при чему се поље `data` разликује по садржају у зависности од врсте догађаја. Овакав приступ поједностављује имплементацију, али значи да формат поруке није формално описан типом током компајлирања (енг. *compile-time*), већ се провера садржаја врши тек на страни клијента, у JavaScript коду.

```
struct BroadcastEnvelope {
    from: Uuid,
    from_email: String,
    data: serde_json::Value,
}
```

Листинг 9: Заједничка омотница за поруке које Session Gateway Service шаље клијентима.

5.3 Code Execution Service

Code Execution Service извршава код који корисници заједно напишу унутар привременог, изолованог Docker контејнера, преко bollard библиотеке која комуницира са Docker демоном локалне машине путем Unix socket-а.

Подржана су само два програмска језика: Python (слика `python:3.12-slim`) и JavaScript (слика `node:20-slim`). Слика се преузима, уколико већ није локално присутна, позивом `/execute`.

Контејнер се покреће са дефинисаним ограничењима меморије, процесора и мреже (табела 7), помоћу HostConfig структуре библиотеке bollard.

Табела 7: Ограничења наметнута контејнеру за извршавање кода.

Ограничење	Вредност
Меморија	128 MB (memory)
Процесор	50% једног језгра (<code>cpu_quota = 50000</code> , <code>cpu_period = 100000</code>)
Мрежа	искључена (<code>network_mode: "none"</code>)
Максимална величина кода	64 KB
Подразумевано / максимално време извршавања	10 s / 30 s (<code>timeout_seconds</code> се ограничава на клијентову вредност, највише 30 s)

Важно је напоменути да су имплементирана искључиво наведена ограничења. Контејнер се извршава без додатних безбедносних мера попут система датотека само за читање, уклањања Linux *capability*-ја, ограничења броја процеса (`pids_limit`) или извршавања под корисничким налогом који није `root`. Ова разматрања су детаљније коментарисана у поглављу 6. По истеку времена (`tokio::time::timeout`) или по завршетку извршавања, контејнер се безусловно уклања, тако да контејнери никада не остају некоришћени у систему.

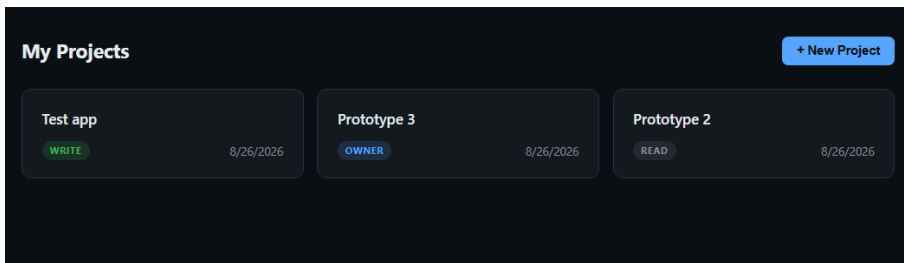
Пошто Code Execution Service управља Docker демоном *host* машине (преко монтираног `/var/run/docker.sock`, видети поглавље 3), а не покреће контејнере унутар

сопственог, изолованог Docker окружења, компромитовање овог сервиса би у пракси значило потпуну контролу над свим контејнерима на хосту, укључујући и остале сервисе система.

5.4 Клијентска апликација

Frontend апликација је реализована у React 18 библиотеци, уз Vite као *development server* и *build* алат. Апликација нема посебну библиотеку за рутирање; прелазак између пријаве, листе пројеката и едитора се решава условним рендеровањем на основу тога да ли корисник има активну сесију (токен сачуван у `localStorage`) и да ли је изабрао пројекат.

Након успешне пријаве, кориснику се приказује почетни екран са листом пројеката (Слика 3), одакле може одабрати постојећи радни простор или креирати нови.



Слика 3: Екран клијентске апликације са листом пројеката корисника.

5.4.1 Едитор кода

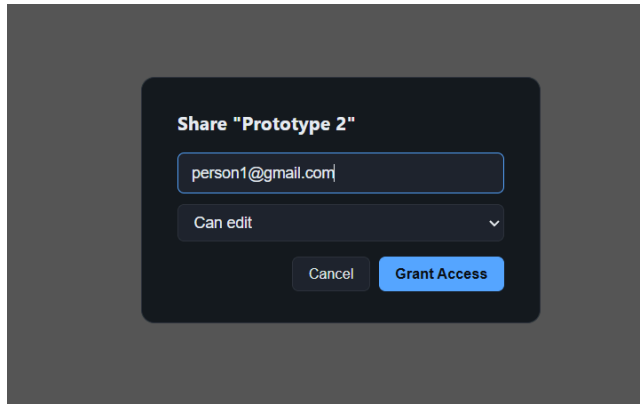
За унос и приказ кода користи се CodeMirror 6, интегрисан преко `@uiw/react-codemirror` библиотеке, са подршком за синтаксно бојење Python и JavaScript кода. Позиције курсора осталих корисника се приказују помоћу CodeMirror StateField механизма. За сваког удаљеног корисника се одржава запис (позиција, боја, ознака) који се приказује као обојена вертикална линија са пливајућом (енг. *floating*) ознаком имејл адресе; позиције курсора се аутоматски усклађују са изменама текста преко `ChangeSet.mapPos` механизма CodeMirror-а, тако да курсор остаје на исправном месту и након што други корисник уметне или обрише карактере испред њега.

Локалне измене се шаљу серверу секвенцијално; клијент чека потврду (одговор на претходно послату операцију) пре него што пошаље следећу операцију из реда чекања, чиме се поједностављује усклађивање локалног приказа са потврђеним стањем документа, на рачун нешто већег перципираног кашњења приликом врло брзог куцања.

5.4.2 Извршавање кода и остале функционалности

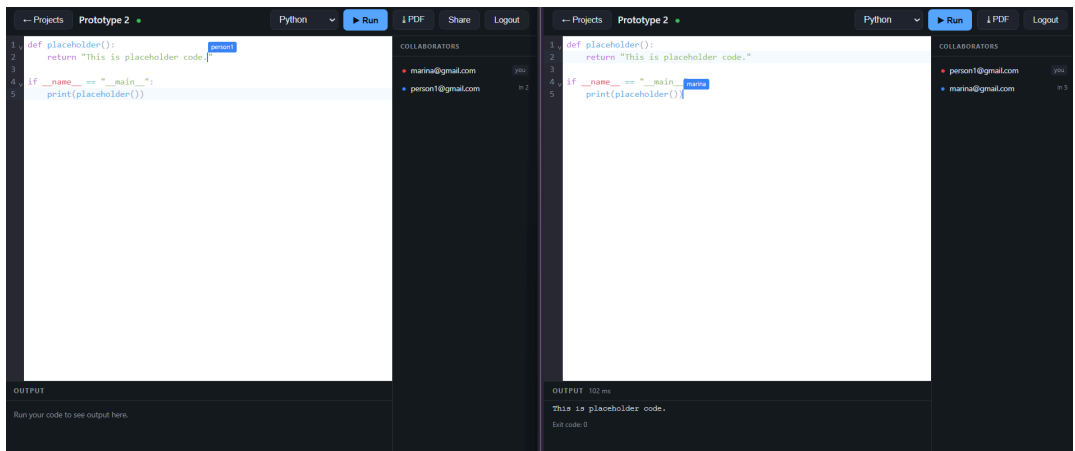
Резултат извршавања кода (`stdout`, `stderr`, излазни код и трајање извршавања) се приказује у бочном панелу. Поред основне функционалности колаборативног уре-

ђивања и извршавања кода, апликација нуди и извоз тренутног садржаја едитора у PDF документ, као и дељење пројекта са другим корисницима избором имејл адресе и жељене улоге приступа. Управљање правима приступа обавља се кроз наменски дијалог (Слика 4), који је интегрисан са Auth & User сервисом.



Слика 4: Дијалог за дељење пројекта, путем којег власник уноси имејл адресу и додељује улогу (читање или писање) новом сараднику.

На слици 5 приказан је кориснички интерфејс апликације током заједничког рада два корисника, где се јасно виде њихови курсори, панел са резултатима извршавања кода, као и опције доступне власнику пројекта.



Слика 5: Упоредни приказ клијентске апликације током колаборативне сесије. Лево: Поглед власника пројекта (са доступним дугметом за дељење) и приказ позиције курсора другог корисника. Десно: Поглед гостујућег корисника након извршавања Python кода, са резултатима приказаним у доњем панелу.

Глава 6

Тестирање и евалуација система

Ово поглавље приказује начин на који је проверавана исправност функционалности система CodeStream током развоја, кроз функционалне сценарије ручног тестирања, као и налазе безбедносног прегледа кода спроведеног над имплементацијом описаном у поглављима 4 и 5. На крају поглавља су сумирана ограничења тренутног приступа тестирању система.

6.1 Приступ тестирању

Систем тренутно не садржи аутоматизоване тестове, нити јединичне тестове (енг. *unit tests*) над RGA алгоритмом или HTTP руковаоцима, ни интеграционе тестове који би покретали више сервиса заједно. Провера исправности функционалности је, због тога, вршена ручно, покретањем целог система помоћу Docker Compose-а и извршавањем сценарија описаних у наставку кроз више истовремено отворених сесија прегледача.

Одсуство аутоматизованих тестова представља реалан недостатак тренутне имплементације: свака измена изворног кода носи ризик да неприметно „поквари“ неко од постојећих понашања, а исправност RGA алгоритма, где грешке у поређењу идентификатора или у одређивању позиције уметања могу произвести суптилно погрешан, а не очигледно неисправан резултат, нарочито добија на значају код кода који се ослања искључиво на ручно тестирање. Ово ограничење је детаљније размотрено у одељку 6.4 и поглављу 7.

6.2 Функционални сценарији

Табела 8 сумира главне функционалне сценарије који су ручно проверени током развоја система.

Табела 8: Ручно тестирани функционални сценарији.

Функционалност	Проверено понашање
Регистрација и пријава	Успешна регистрација новог корисника; одбијање регистрације са већ заузетом имејл адресом (HTTP 409); одбијање пријаве са погрешном лозинком.
Контрола приступа пројекту	Власник пројекта има пуна права; корисник са улогом write може да уређује

Функционалност	Проверено понашање
	документ; корисник са улогом read може да прати измене у реалном времену, али његови покушаји уређивања бивају одбачени од стране Session Gateway Service-a; корисник без икаквог приступа не може да успостави WebSocket везу са пројектом.
Истовремено уређивање	Отварањем истог пројекта у две или више независних сесија прегледача (различити кориснички налози), проверена је конвергенција текста након низа истовремених уметања и брисања на истим и на суседним позицијама, у складу са очекиваним понашањем RGA алгорита описаним у поглављу 4.
Позиције курсора и промена језика	Позиција курсора сваког корисника се исправно приказује осталим учесницима сесије и исправно се помера при туђим изменама текста испред ње; промена изабраног програмског језика се преноси свим учесницима.
Извршавање кода	Проверено извршавање исправног Python и JavaScript кода (тачан stdout); прикупљање stderr-а код кода који баца изузетак; прекид извршавања и одговарајућа порука по истеку времена; одбијање захтева за неподржан програмски језик и за код који прелази дозвољену величину.
Прекид и поновно успостављање везе	Затварање картице прегледача или прекид мреже резултује уклањањем корисника из „собе“ на страни Session Gateway Service-a (ослобађање ресурса) и обавештавањем осталих учесника да је корисник напустио сесију.

6.3 Безбедносна разматрања

Прегледом изворног кода уочена су два пропуста у контроли приступа, као и делимична, а не потпуна, изолација контејнера за извршавање кода. Ови налази нису искоришћени за нарушавање система, већ су документовани као основа за будуће исправке:

- *Endpoint* GET /documents/:id/state (CRDT Sync Service, поглавље 4) захтева важећи JWT токен, али не проверава да ли пријављени корисник заиста има одобрен приступ траженом пројекту; било који пријављени корисник који погоди или сазна UUID пројекта може да преузме комплетан садржај карактера документа, укључујући и tombstone карактере који нису видљиви у самом едитору.
- *Endpoint* POST /execute (Code Execution Service) захтева важећи JWT токен, али не проверава да ли је извршавање кода везано за пројекат за који корисник има одобрен приступ; сваки пријављени корисник система може да изврши произвољан код, независно од чланства у било ком пројекту.
- *Sandbox* контејнер за извршавање кода ограничава меморију, процесорско време и мрежни приступ (поглавље 5, табела 7), али се извршава са подразумеваним корисником базне слике (у пракси root унутар контејнера), без датотечног система само за читање, без уклањања Linux *capability*-ја и без ограничења броја процеса. Према препорукама за безбедно коришћење контејнера [16], ове мере представљају додатни, независан слој одбране (енг. *defense in depth*) који тренутна имплементација не користи у потпуности, иако су најкритичнија ограничења (меморија, процесор, мрежа) присутна.

Ниједан од ова два пропуста у контроли приступа не омогућава приступ подацима других пројеката осим оних чији идентификатор нападач већ познаје или погоди (UUID простор чини масовно „погађање“ непрактичним), али оба представљају одступање од принципа најмање привилегије и требало би их отклонити пре евен-туалног постављања система изван контролисаног, демонстрационог окружења.

6.4 Ограничења приступа тестирању

Тренутни приступ тестирању има три главна ограничења:

1. **Одсуство аутоматизованих тестова** – ручно тестирање не скалира се добро са растом кодне базе и не штити поуздано од регресија приликом будућих измена.
2. **Одсуство тестирања оптерећења** – понашање система под већим бројем истовремених корисника (нпр. 50 или 100 корисника у истом пројекту) није мерено; линеарна сложеност уметања карактера (поглавље 4) сугерише да би перформансе опадале са дужином документа и учесталошћу измена, али ова хипотеза није емпиријски потврђена мерењем.

3. **Тестирање ограничено на један развојни рачунар** – сви сценарији су проверени покретањем целог система локално, путем Docker Compose-a, без провере понашања система у окружењу са стварно раздвојеним мрежним кашњењем између компоненти.

Ова ограничења, заједно са предлозима за њихово отклањање, детаљније су размотрена у поглављу [7](#).

У раду је приказано пројектовање и имплементација система CodeStream, дистрибуираног система за колаборативно уређивање програмског кода у реалном времену, заснованог на CRDT структури података. Систем омогућава да више корисника истовремено уређује исти документ, при чему се конфликти настали услед истовремених измена решавају применом RGA алгоритма (поглавље 4), уместо централизоване трансформације операција карактеристичне за ОТ приступ (поглавље 2). Поред самог механизма синхронизације, реализована је и микросервисна архитектура (поглавље 3) која обухвата управљање идентитетом и правима приступа, посредовање у *real-time* комуникацији путем WebSocket протокола и безбедно, изоловано извршавање кода унутар Docker контејнера (поглавље 5).

7.1 Осврт на остварене резултате

Основни циљ рада је остварен: RGA алгоритам исправно разрешава конфликти приликом истовременог уметања и брисања карактера, гарантујући конвергенцију свих учесника ка истом коначном садржају документа, без обзира на редослед обраде операција (одељак 4.3.1). Реализована микросервисна архитектура јасно раздваја надлежности система (идентитет, синхронизацију документа, посредовање у комуникацији и извршавање кода), а асинхрона комуникација путем RabbitMQ-а раздваја пријем корисничких измена од њихове обраде на CRDT сервису.

Истовремено, поглавље 6 идентификовало је неколико конкретних недостатака тренутне имплементације: одсуство контроле приступа код два *endpoint*-а (GET / documents/:id/state и POST /execute), делимичну изолацију *sandbox* контејнера и потпуно одсуство аутоматизованих тестова. Ови налази не умањују остварени резултат у погледу примене CRDT приступа, али јасно указују на кораке неопходне пре евентуалног постављања система изван контролисаног, демонстрационог окружења.

7.2 Ограничења решења

Поред безбедносних налаза из поглавља 6, тренутна имплементација RGA алгоритма (поглавље 4) има три структурна ограничења:

1. **Трајно чување *tombstone* карактера.** Обрисани карактери се никада не уклањају из низа нити из стања сачуваног у Redis бази, због чега структура документа

непрекидно расте и код других сесија интензивног уређивања, чак и када обрисани садржај више никада неће бити приказан кориснику.

2. **Линеарна сложеност уметања.** Проналажење позиције претходног карактера и померање елемената низа при сваком уметању захтева време пропорционално укупном броју карактера у документу (укључујући *tombstone* карактере), што поставља горњу границу перформанси за веома велике документе или за документе са дугом историјом измена.
3. **Централизовани логички сат.** Идентификатори карактера се додељују секвенцијално, унутар једне активне инстанце CRDT Sync Service-а по документу, а не независно на страни сваке реплике уз накнадно усклађивање. Овим се поједностављује имплементација, али систем у тренутном облику не подржава рад више независних реплика (нпр. рад без сталне везе са сервером).

7.3 Правци даљег развоја

На основу уочених ограничења, могући правци даљег развоја система обухватају:

- **Управљање *tombstone* карактерима.** Увођење периодичног поступка прикупљања отпадака (енг. *garbage collection*) који би безбедно уклањао *tombstone* карактере за које је, на основу потврда свих активних учесника сесије да су „видели“ операцију брисања, извесно да их ниједна будућа операција више неће референцирати. Паралелно, вредело би испитати алтернативне структуре попут YATA [10] или RGA-Split приступа засноване на груписању суседних карактера у блокове, ради смањења утицаја линеарне сложености уметања, уз мерење латенце конвергенције при већем броју истовремених корисника.
- **Подршка за рад без сталне везе (*offline-first*).** Локално чување непотврђених операција на страни клијента и њихово слање по поновном успостављању везе, уз делта синхронизацију при поновном повезивању, што би захтевало и прелазак са тренутног централизованог логичког сата на модел у ком клијенти самостално додељују идентификаторе новим карактерима.
- **Чување историје измена.** Трајно чување сваке потврђене CRDT операције (нпр. у PostgreSQL бази, као непроменљив низ догађаја), што би омогућило преглед историје измена по кориснику, враћање документа на претходно стање и потенцијално грађање верзија документа.
- **Отклањање уочених безбедносних пропуста.** Увођење провере права приступа пројекту код *endpoint*-а GET /documents/:id/state и POST /execute, као и проширење *sandbox* конфигурације датотечним системом само за читање, уклањањем непотребних Linux *capability*-ја и извршавањем под корисничким налогом који није root [16].

- **Увођење аутоматизованих тестова.** Јединични тестови над RGA алгоритмом (нарочито над функцијом уметања карактера и разрешавањем конфликта) и интеграциони тестови над HTTP и WebSocket рутама сервиса, чиме би се смањио ризик од регресија приликом будућих измена и омогућило мерење перформанси под контролисаним оптерећењем.

Систем CodeStream, у облику приказаном у овом раду, представља функционалну основу која демонстрира практичну примену CRDT структура података у контексту колаборативног уређивања кода, а наведени правци даљег развоја представљају природан наставак рада ка систему спремној за употребу изван демонстрационог окружења.

Списак слика

Слика 1	Архитектура система CodeStream.	7
Слика 2	Ток обраде CRDT операције кроз систем.	17
Слика 3	Екран клијентске апликације са листом пројеката корисника.	24
Слика 4	Дијалог за дељење пројекта, путем којег власник уноси имејл адресу и додељује улогу (читање или писање) новом сараднику.	25
Слика 5	Упоредни приказ клијентске апликације током колаборативне сесије. Лево: Поглед власника пројекта (са доступним дугметом за дељење) и приказ позиције курсора другог корисника. Десно: Поглед гостујућег корисника након извршавања Python кода, са резултатима приказаним у доњем панелу.	25

Списак листинга

Листинг 1	Идентификатор карактера CharId и релација уређења над њим.	13
Листинг 2	Структуре RgaChar, RgaDocument и операције које стижу од клијента. .	14
Листинг 3	Алгоритам уметања карактера у RGA документ.	15
Листинг 4	Означавање карактера као обрисаног (<i>tombstone</i>).	17
Листинг 5	Шема релационе базе података – корисници, пројекти и права приступа.	19
Листинг 6	Хеширање лозинке приликом регистрације корисника.	20
Листинг 7	Садржај JWT токена (Claims).	20
Листинг 8	Дељено стање AppState сервиса Session Gateway.	21
Листинг 9	Заједничка омотница за поруке које Session Gateway Service шаље клијентима.	23

Списак табела

Табела 1	Приступи усклађивању истовремених измена код изабраних алата за колаборативно уређивање.	5
Табела 2	Преглед сервиса система CodeStream.	8
Табела 3	Мапирање HTTP рута на сервисе у оквиру nginx конфигурације.	10
Табела 4	Преглед технолошког стека система CodeStream.	10
Табела 5	Разрешавање конфликта код два уметања на истој позицији.	16
Табела 6	HTTP руте сервиса Auth & User Service.	21
Табела 7	Ограничења наметнута контејнеру за извршавање кода.	23
Табела 8	Ручно тестирани функционални сценарији.	27

Списак коришћених скраћеница

Скраћеница	Опис
AMQP	Advanced Message Queuing Protocol (протокол за размену порука коришћен од стране RabbitMQ-a)
API	Application Programming Interface (апликациони програмски интерфејс)
CRDT	Conflict-free Replicated Data Type (структура података без конфликта за реплицирано усклађивање)
HS256	HMAC with SHA-256 (симетрични алгоритам за криптографско потписивање података, попут JWT токена, помоћу једног дељеног тајног кључа).
HTTP	HyperText Transfer Protocol (протокол за пренос хипертекста)
JSON	JavaScript Object Notation (формат за размену података)
JWT	JSON Web Token (сигурносни токен заснован на JSON формату)
OT	Операциона трансформација (енг. <i>Operational Transformation</i>)
PDF	Portable Document Format (преносиви формат документа)
RGA	Replicated Growable Array (реплицирани растући низ – CRDT структура коришћена за синхронизацију документа)
SPA	Single Page Application (апликација са једном страном)
URL	Uniform Resource Locator (јединствени идентификатор и локатор ресурса)
UUID	Universally Unique Identifier (универзално јединствени идентификатор)
WS	WebSocket (протокол који омогућава двосмерну комуникацију у реалном времену преко једне дуготрајне TCP конекције)

Списак коришћених појмова

Појам	Објашњење
Tombstone	Карактер логички означен као обрисан, али физички задржан у структури документа јер га каснија операција уметања може референцирати као претходника.
Лампоров логички сат	Бројач који сваком догађају додељује вредност тако да се, ако један догађај узрокује други, првом догађају гарантовано додели мања вредност.
Конвергенција	Својство да све реплике дељеног документа, након примене истог скупа операција, доспевају до истог коначног стања, без обзира на редослед примене.
Sandbox	Изоловано извршно окружење (у овом раду – Docker контејнер) са ограниченим ресурсима и правима приступа, намењено безбедном извршавању непоузданог кода.
Reverse proxy	Сервер који прима све долазне мрежне захтеве и прослеђује их одговарајућем унутрашњем сервису, представљајући јединствену тачку приступа систему.
Broadcast	Слање исте поруке свим тренутно активним конекцијама у оквиру исте „собе“ (пројекта).
Read-Only	Режим рада у коме су подаци само за читање, без могућности измене.
Трка (race condition)	Ситуација у којој исход зависи од непредвидивог редоследа извршавања конкурентних операција над дељеним ресурсом.
Docker-outside-of-Docker	Образац у ком сервис, преко монтираног Unix socket-а, управља Docker демоном хост машине, уместо да покреће сопствене, угнежђене контејнере.
Health check	Периодична провера да ли је одређена компонента система спремна да прима захтеве, коришћена за координисано покретање зависних сервиса.
Argon2id	Савремени криптографски алгоритам за хеширање лозинки, дизајниран да буде високо отпоран на нападе помоћу графичких процесора (GPU) и нападе бочним каналима (енг. side-channel attacks).
Unix socket	(енг. Unix domain socket) Крајња тачка за међупроцесну комуникацију (IPC) која омогућава ефикасну размену података између процеса који се извршавају на истом оперативном систему, без напуштања језгра оперативног система (коришћено за брзу комуникацију са Docker демоном).

Биографија

Марина Ивановић је рођена 23. марта 2003. године у Ужицу. У Пожеги је завршила гимназију „Свети Сава“ 2022. године. Након завршене средње школе, уписује Факултет техничких наука Универзитета у Новом Саду, студијски програм Софтверско инжењерство и информационе технологије. Све испите предвиђене планом и програмом полаже са просечном оценом 9,42. Током студија се додатно усавршавала кроз практичне пројекте и учешће на стручним такмичењима, попут такмичења „5 Days in Cloud“. Њена главна стручна интересовања обухватају развој позадинских софтверских система, архитектуру микросервиса, дизајн конкурентних система и рад са инфраструктуром у облаку. Поред тога, активно се занима за развој пуног стека веб апликација и технологије за контејнеризацију софтвера.

Литература

- [1] G. D. Team, “What’s Different About the New Google Docs: Conflict Resolution.” Accessed: July 22, 2026. [Online]. Доступно на https://drive.googleblog.com/2010/09/whats-different-about-new-google-docs_22.html
- [2] E. Wallace, “How Figma’s Multiplayer Technology Works.” Accessed: July 22, 2026. [Online]. Доступно на <https://www.figma.com/blog/how-figmas-multiplayer-technology-works/>
- [3] A. Silver, “Introducing Visual Studio Live Share.” Accessed: July 22, 2026. [Online]. Доступно на <https://code.visualstudio.com/blogs/2017/11/15/live-share>
- [4] C. A. Ellis and S. J. Gibbs, “Concurrency Control in Groupware Systems,” in *Proceedings of the 1989 ACM SIGMOD International Conference on Management of Data*, Portland, Oregon, USA: ACM, 1989, pp. 399–407. doi: [10.1145/67544.66963](https://doi.org/10.1145/67544.66963).
- [5] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “A Comprehensive Study of Convergent and Commutative Replicated Data Types,” technical report RR-7506, 2011. [Online]. Доступно на <https://inria.hal.science/inria-00555588>
- [6] H.-G. Roh, M. Jeon, J.-S. Kim, and J. Lee, “Replicated Abstract Data Types: Building Blocks for Collaborative Applications,” *Journal of Parallel and Distributed Computing*, vol. 71, no. 3, pp. 354–368, 2011, doi: [10.1016/j.jpdc.2010.12.006](https://doi.org/10.1016/j.jpdc.2010.12.006).
- [7] G. Oster, P. Urso, P. Molli, and A. Imine, “Data Consistency for P2P Collaborative Editing,” in *Proceedings of the 2006 ACM Conference on Computer Supported Cooperative Work (CSCW ’06)*, Banff, Alberta, Canada: ACM, 2006, pp. 259–268. doi: [10.1145/1180875.1180916](https://doi.org/10.1145/1180875.1180916).
- [8] L. Lamport, “Time, Clocks, and the Ordering of Events in a Distributed System,” *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978, doi: [10.1145/359545.359563](https://doi.org/10.1145/359545.359563).
- [9] B. Nédelec, P. Molli, A. Mostéfaoui, and E. Desmontils, “LSEQ: An Adaptive Structure for Sequences in Distributed Collaborative Editing,” in *Proceedings of the 2013 ACM Symposium on Document Engineering (DocEng ’13)*, Florence, Italy: ACM, 2013, pp. 37–46. doi: [10.1145/2494266.2494278](https://doi.org/10.1145/2494266.2494278).
- [10] P. Nicolaescu, K. Jahns, M. Derntl, and R. Klamma, “Near Real-Time Peer-to-Peer Shared Editing on Extensible Data Types,” in *Proceedings of the 19th ACM Conference*

on Computer Supported Cooperative Work and Social Computing Companion (GROUP '16), ACM, 2016, pp. 39–49. doi: [10.1145/2957276.2957310](https://doi.org/10.1145/2957276.2957310).

- [11] K. Jahns, “Yjs: Shared Data Types for Building Collaborative Software.” Accessed: July 22, 2026. [Online]. Доступно на <https://github.com/yjs/yjs>
- [12] M. Kleppmann and A. R. Beresford, “A Conflict-Free Replicated JSON Datatype,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 28, no. 10, pp. 2733–2746, 2017, doi: [10.1109/TPDS.2017.2697382](https://doi.org/10.1109/TPDS.2017.2697382).
- [13] M. Kleppmann, “Automerge: A JSON-like Data Structure that is Automatically Synced Between Devices.” Accessed: July 22, 2026. [Online]. Доступно на <https://github.com/automerge/automerge>
- [14] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. O'Reilly Media, 2021.
- [15] O. Foundation, “Password Storage Cheat Sheet.” Accessed: July 22, 2026. [Online]. Доступно на https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- [16] D. Inc., “Docker Engine Security.” Accessed: July 22, 2026. [Online]. Доступно на <https://docs.docker.com/engine/security/>